

Unit:9

DISTRIBUTED OPERATING SYSTEM

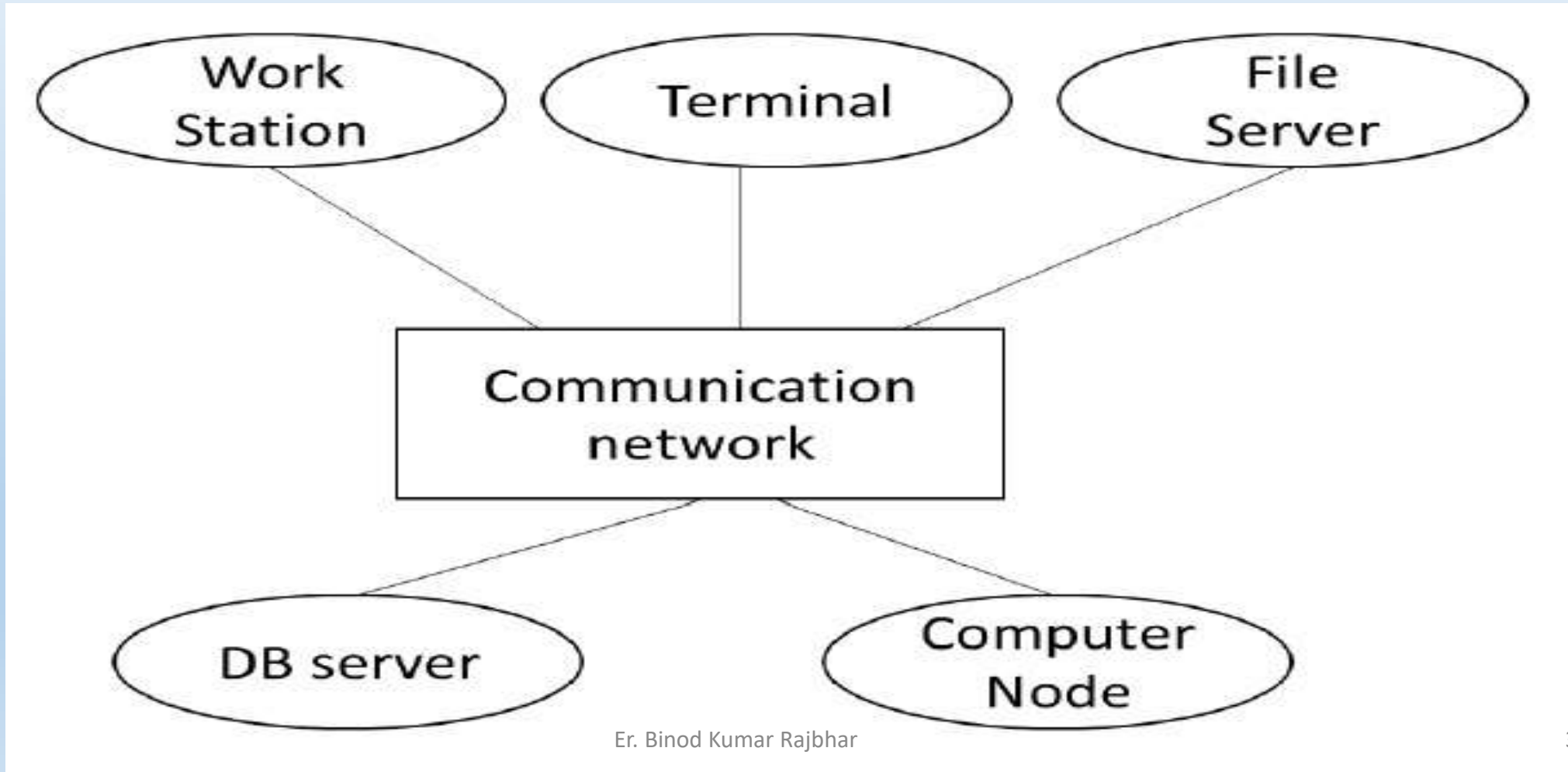
Prepared by

Er. Binod Kumar Rajbhar

Distributed Operating System

- ✓ A distributed operating system is an important type of operating system.
- ✓ An operating system is basically, a program that acts as an interface between the system hardware and the user.
- ✓ Moreover, it handles all the interactions between the software and the hardware.
- ✓ A distributed operating system is one in which several computer systems connect through a single communication channel.
- ✓ Moreover, these systems have their individual processors and memory. Furthermore, these processors communicate through **high-speed buses or telephone lines**.

The distributed operating system is depicted below –



Features/Characteristics of Distributed Operating System

1. Resource Sharing

The main important feature of this system is that it allows users to share resources. Moreover, they can share resources in a secure and controlled manner. Resources can be of any type. For example, some common resources which are shared can be printers, files, data, storage, web pages, etc.

2. Openness

This means that the services which the system provides are openly displayed through interfaces. Moreover, these interfaces provide only the syntax of the services. For example, the type of functions, their return types, parameters, etc. These interfaces use Interface Definition Languages (IDL).

3. Concurrency

It means that several tasks take place at different nodes of the system simultaneously. Moreover, these tasks can also interact with each other. It results in increasing the efficiency of the system.

4. Scalability

It refers to the fact that the efficiency of the system should not change when more nodes are added to the system. Moreover, the performance for the system with 100 nodes should be equal to the system with 1000 nodes.

5. Fault Tolerance

It means that the user can still work with the system in the case, hardware, or software fails.

6. Transparency

The main goal of a distributed OS is to hide the fact that the resources are being shared. Furthermore, transparency means that the user should not know that the resources he is using are shared. Moreover, for the user, the system should be a separate individual unit.

Advantages of distributed operating systems

- ✓ Give more performance than single system
- ✓ If one pc in distributed system malfunction or corrupts then other node or pc will take care of
- ✓ More resources can be added easily
- ✓ Resources like printers can be shared on multiple pc's

Examples of distributed operating systems:-

- Windows server 2003
- Windows server 2008
- Windows server 2012
- Ubuntu
- Linux (Apache Server)

Disadvantages of distributed operating systems

- ✓ Security problem due to sharing
- ✓ Some messages can be lost in the network system
- ✓ Bandwidth is another problem if there is large data then all network wires to be replaced which tends to become expensive
- ✓ Overloading is another problem in distributed operating systems
- ✓ If there is a database connected on local system and many users accessing that database through remote or distributed way then performance become slow
- ✓ The databases in network operating is difficult to administrate then single user system

Advantage of Distributed Systems over Centralized

- 1. Speed.** When used to implement parallel processing where only goal is to achieve maximum speed on a single problem, distributed systems can achieve very high speed as compared to the centralized ones.
- 2. Inherent Distribution-** Another reason for building a distributed system is that some applications are inherently distributed. Banking, Airline reservation etc. are examples of the applications that are inherently distributed. When all the branches of a bank are connected, we have a commercial distributed system.
- 3. Reliability-** Another potential advantage of a distributed system over a centralized one is higher reliability. By distributing the workload over many machines, a single chip failure will bring down at most one machine, leaving the rest intact. Ideally, if 5 percent of the machines are down at any moment, the system should be able to continue to work with a 5 percent loss in performance. For critical applications, such as control of nuclear reactors or aircraft, using a distributed system to achieve high reliability may be a dominant consideration

4. Incremental Growth. Finally, incremental growth is also potentially a big plus. Often a company will buy a mainframe with the intention of doing all its work on it. If the company prospers and the workload grows, at a certain point the mainframe will no longer be adequate. The only solutions are to either replace the mainframe with a larger one (if it exists), or add a second mainframe. Both of these can cause management difficulties with the company's operations. In contrast, with a distributed system, it may be possible to simply add more processors to the system, thus allowing it to expand gradually as the need arises.

Advantages of Distributed System over Independent PCs

- ❖ **Data Sharing:** It is always possible to have people working independently on their own PCs, for the same business. But we know that users may need to share data. For example, airline reservation clerks need access to the master database of flights and existing reservations. Giving each clerk his own private copy Of the entire database would not work, since nobody would be able to know which seats the other clerks have all ready sold. Shared data is absolutely essential to this of application and so, the machines must be interconnected
- ❖ **Device Sharing:** Data are not the only things to be shared. Expensive . peripherals such as color laser printers, photo-type setters and massive archival storage devices are also among the few things that should be sharable.

3. Communication: Another reason to connect a group of isolated computers into a distributed system is to achieve enhanced person-to-person communication. For many people, electronic mail has numerous attractions over paper mail, telephone and FAX as it makes various kinds of documents accessible by the people without changing their formats. This requires a good communication setup.

4. Flexibility: Finally, a distributed System is potentially more flexible than giving each user a isolated personal computer. If, we have a mixture of personal and shared computers, perhaps of different sizes, and jobs run on the most appropriate ones rather than always on the owners machine. Then the workload can be spread Over the computers more effectively and letting people run their jobs elsewhere may compensate for the loss of a few machines. This way the system will be more flexible.

Network Operating System

- ✓ Network operating system(NOS) is software that connects multiple devices and computers on the network and allows them to share resources on the network.
- ✓ **Functions of the NOS :**
Following are the main functions of NOS :
 - Creating and managing user accounts on the network.
 - Controlling access to resources on the network.
 - Provide communication services between the devices on the network.
 - Monitor and troubleshoot the network.
 - Configuring and Managing the resources on the network.

Advantages of Network operating systems :

- Highly stable due to central server.
- Provide good security.
- Upgradation of new technology and hardware can be easily implemented in the network.
- Provide remote access to servers from different locations.

Disadvantages of Network operating systems :

- Depend on the central location to perform the operations.
- High cost to buying server.
- Regular updating and maintenance are required.

Difference Between Network Operating System and Distributed Operating System

Network Operating System's main objective is to provide the local services to remote client.

In Network Operating System, Communication takes place on the basis of files.

Network Operating System is more scalable than Distributed Operating System.

In Network Operating System, fault tolerance is less.

Rate of autonomy in Network Operating System is high.

Ease of implementation in Network Operating System is also high.

In Network Operating System, All nodes can have different operating system.

Distributed Operating System's main objective is to manage the hardware resources.

In Distributed Operating System, Communication takes place on the basis of messages and shared memory.

Distributed Operating System is less scalable than Network Operating System.

While in Distributed Operating System, fault tolerance is high.

While The rate of autonomy in Distributed Operating System is less.

While in Distributed Operating System Ease of implementation is less.

While in Distributed Operating System, All nodes have same operating system.

Hardware and Software Concept in Distributed Operation System

Hardware Concept in Distributed Operation System

- ✓ All distributed systems consist of multiple CPUs.
- ✓ There are several different ways the hardware can be arranged.
- ✓ The important thing related to hardware is that how they are interconnected and how they communicate with each other.
- ✓ It is important to take a deep look at distributed system hardware, in particular, how the machines are connected together and how they interact.
- ✓ Many classification schemes for multiple CPU computer systems have been proposed over the years, but none of them have really implemented.
- ✓ Still, the most commonly used taxonomy is Flynn's (1972), but it was in basic stage.
- ✓ In this scheme, Flynn took only two things to consider i.e. the **number of instruction streams** and the **number of data streams**.

Single Instruction, Single Data Stream (SISD)

- ✓ A computer with a single instruction stream and a single data stream is called **SISD**.
- ✓ All traditional uni-processor computers (i.e., those having only one CPU) fall under this category, from personal computers to large mainframes.
- ✓ SISD flow concept is given in the figure below.

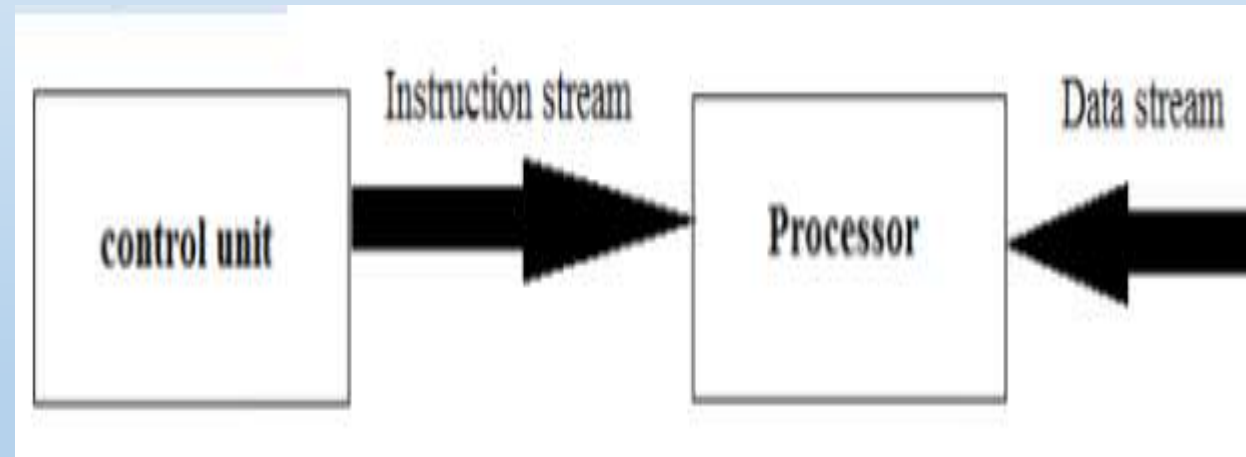


Fig: SISD Flow structure

Single Instruction, Multiple Data Stream (SIMD)

- ✓ The next category is SIMD, i.e. single instruction stream, multiple data stream.
- ✓ This type uses an array of processors with only one instruction unit that fetches an instruction, and multiple data units which work in parallel.
- ✓ These machines are used where there need to apply the same instruction for multiple data example, adding up all the elements of 64 independent vectors.
- ✓ Some supercomputers are SIMD.
- ✓ Figure below shows the SIMD flow structure.

Single Instruction, Multiple Data Stream (SIMD)

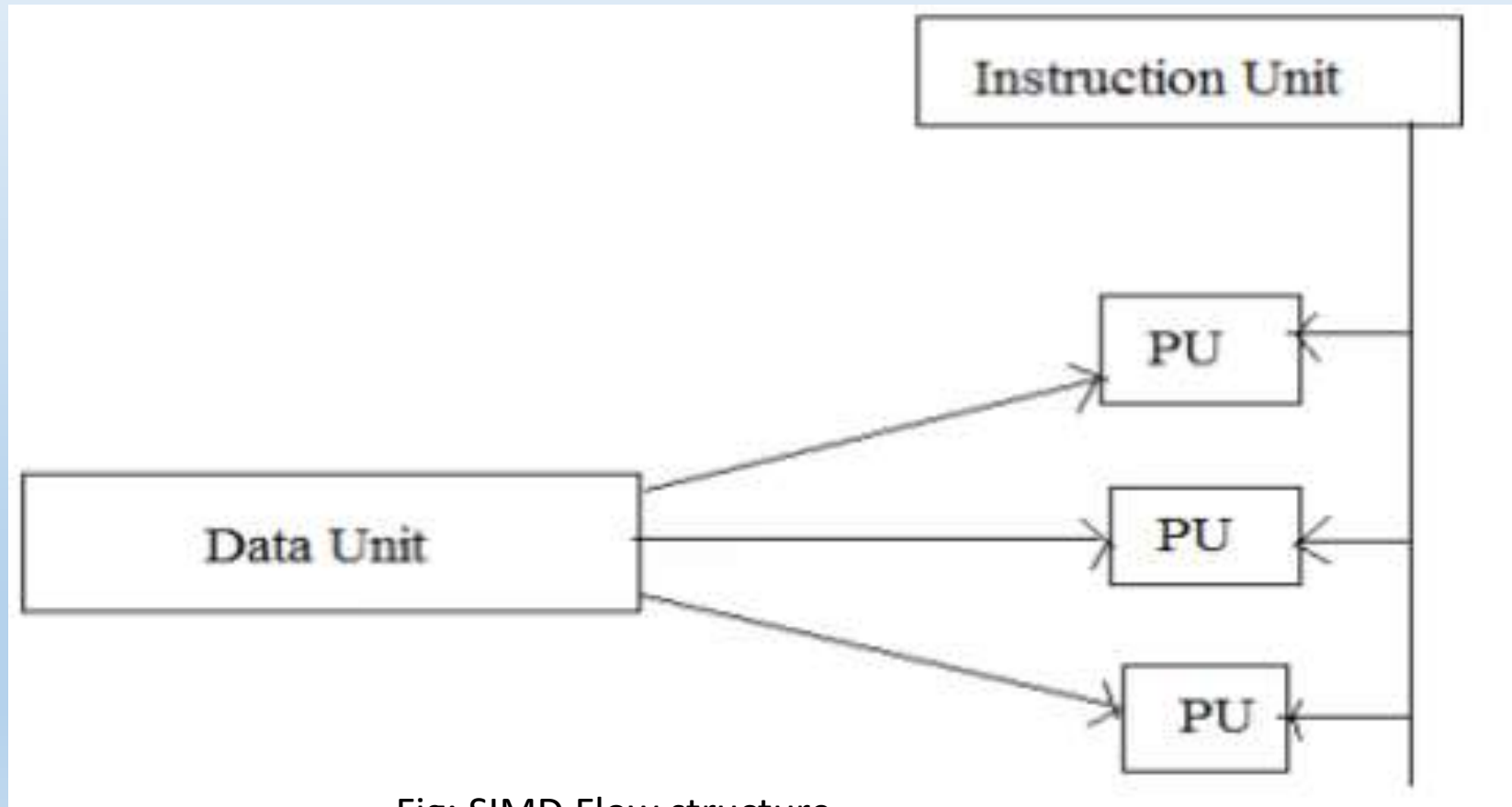
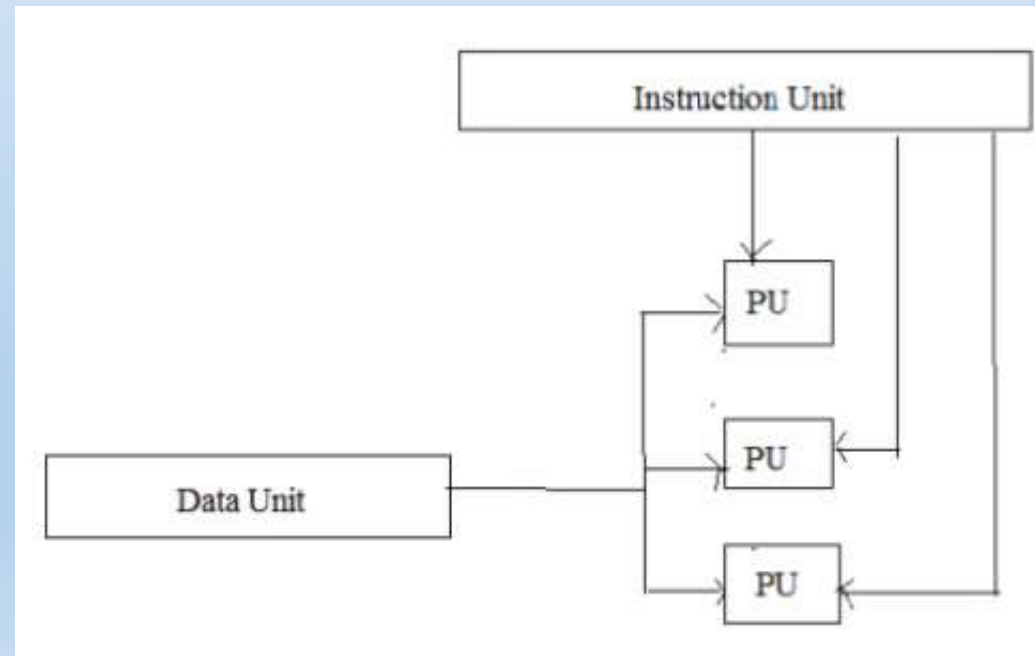


Fig: SIMD Flow structure

Multiple Instruction, Single Data Stream (MISD)

- ✓ The next category is **MISD** i.e. multiple instruction streams, single data stream.
- ✓ This structure was worked when there are multiple different instructions to operate on the same type of data.
- ✓ In general MISD architecture is not use more in practical.

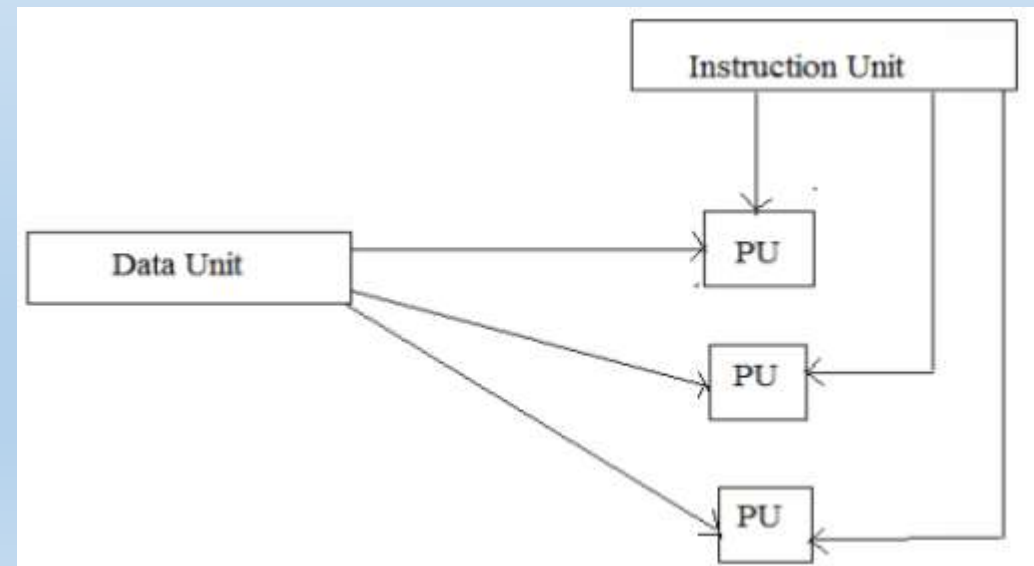
Fig: MISD Flow structure



Multiple Instruction, Multiple Data Stream (MIMD)

- ✓ The next category is MIMD, which has multiple instructions performances on multiple data units.
- ✓ This means a group of independent computers; each has its own program counter, program, and data.
- ✓ All distributed systems are MIMD, so this classification system is not more useful for simple purposes.

Fig: MIMD Flow structure



Software Concepts

- ✓ Although the hardware is important, the software is even more important.
- ✓ The image that a system presents to its users and how they think about the system is largely determined by the operating system software not the hardware.
- ✓ There are two types of operating system namely tightly coupled operating system and loosely couple operating system for multiprocessor and multicomputer.
- ✓ Loosely coupled software allows machines and users of a distributed system to be fundamentally independent of one another.
- ✓ Tightly coupled system consider a multiprocessor dedicated to running a single chess program in parallel.
- ✓ Each CPU is assigned a board to evaluate and it spends its time examining that board and all the boards that can be generated from it.

Communication in Distributed System

- ✓ There are two approaches for communication in distributed system
 - ❖ Shared memory and
 - ❖ Message passing

Shared memory system:

- ✓ Shared memory is the fundamental model of inter process communication.
- ✓ In a shared memory system, in the address space region the cooperating communicate with each other by establishing the shared memory region.
- ✓ Shared memory concept works on fastest inter process communication.
- ✓ If the process wants to initiate the communication and it has some data to share, then establish the shared memory region in its address space.
- ✓ After that, another process wants to communicate and tries to read the shared data, and must attach itself to the initiating process's shared address space.

Message Passing:

- ✓ Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.
- ✓ **For example** – Chat program on the World Wide Web.
- ✓ Message passing provides two operations which are as follows –
 - Send message
 - Receive message
- ✓ Messages sent by a process can be either fixed or variable size.
- ✓ For fixed size messages the system level implementation is straight forward. It makes the task of programming more difficult.
- ✓ The variable sized messages require a more system level implementation but the programming task becomes simpler.
- ✓ If process P1 and P2 want to communicate they need to send a message to and receive a message from each other that means here a communication link exists between them.

The major differences between shared memory and message passing model

Shared Memory

It is one of the region for data communication

It is used for communication between single processor and multiprocessor systems where the processes that are to be communicated present on the same machine and they are sharing common address space.

The shared memory code that has to be read or write the data that should be written explicitly by the application programmer.

It is going to provide a maximum speed of computations because the communication is done with the help of shared memory so system calls are used to establish the shared memory.

In shared memory make sure that the processes are not writing to the same location simultaneously.

It follows a faster communication strategy when compared to message passing technique.

Message Passing

Mainly the message passing is used for communication.

It is used in distributed environments where the communicating processes are present on remote machines which are connected with the help of a network.

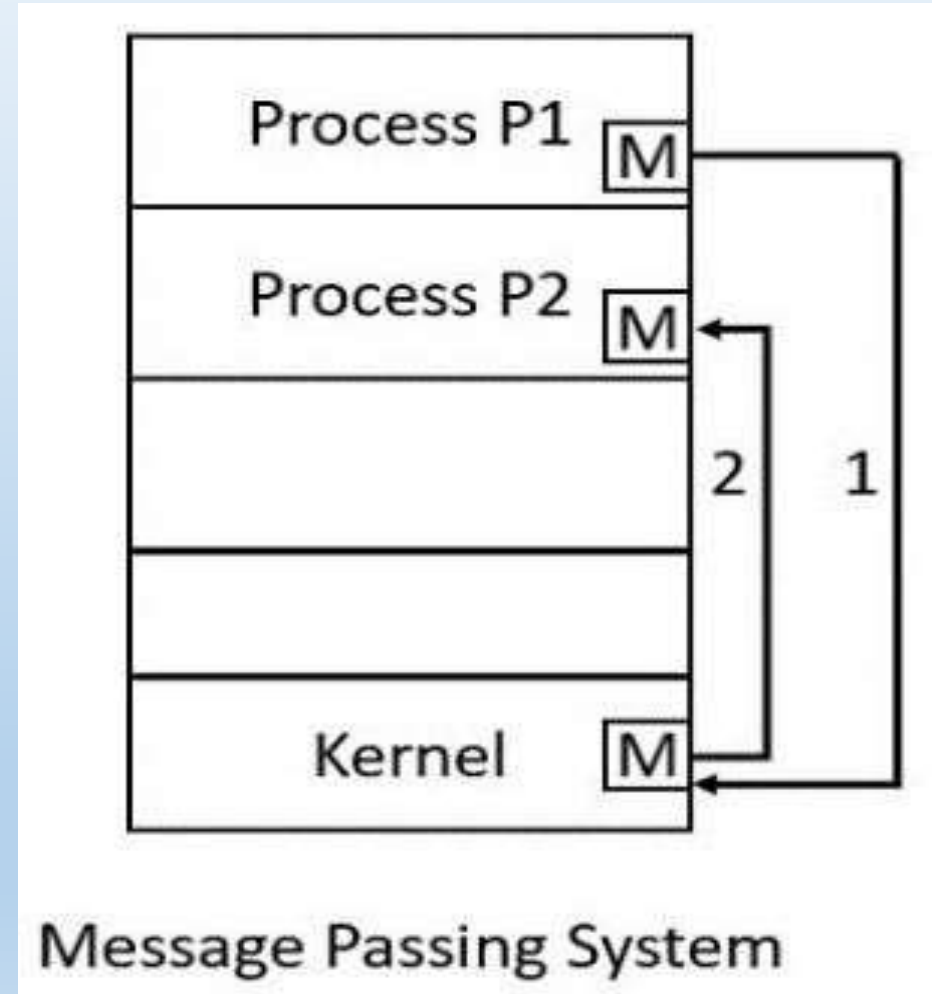
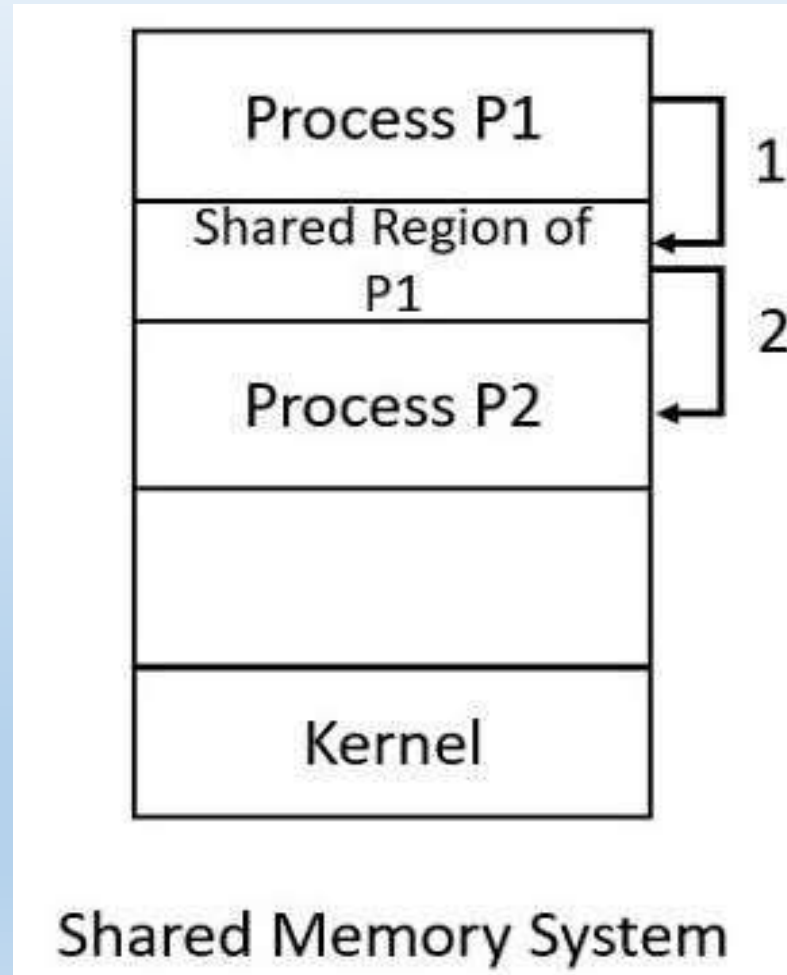
Here no code is required because the message passing facility provides a mechanism for communication and synchronization of actions that are performed by the communicating processes.

Message passing is a time consuming process because it is implemented through kernel (system calls).

Message passing is useful for sharing small amounts of data so that conflicts need not occur.

In message passing the communication is slower when compared to shared memory technique.

Communication in Distributed System



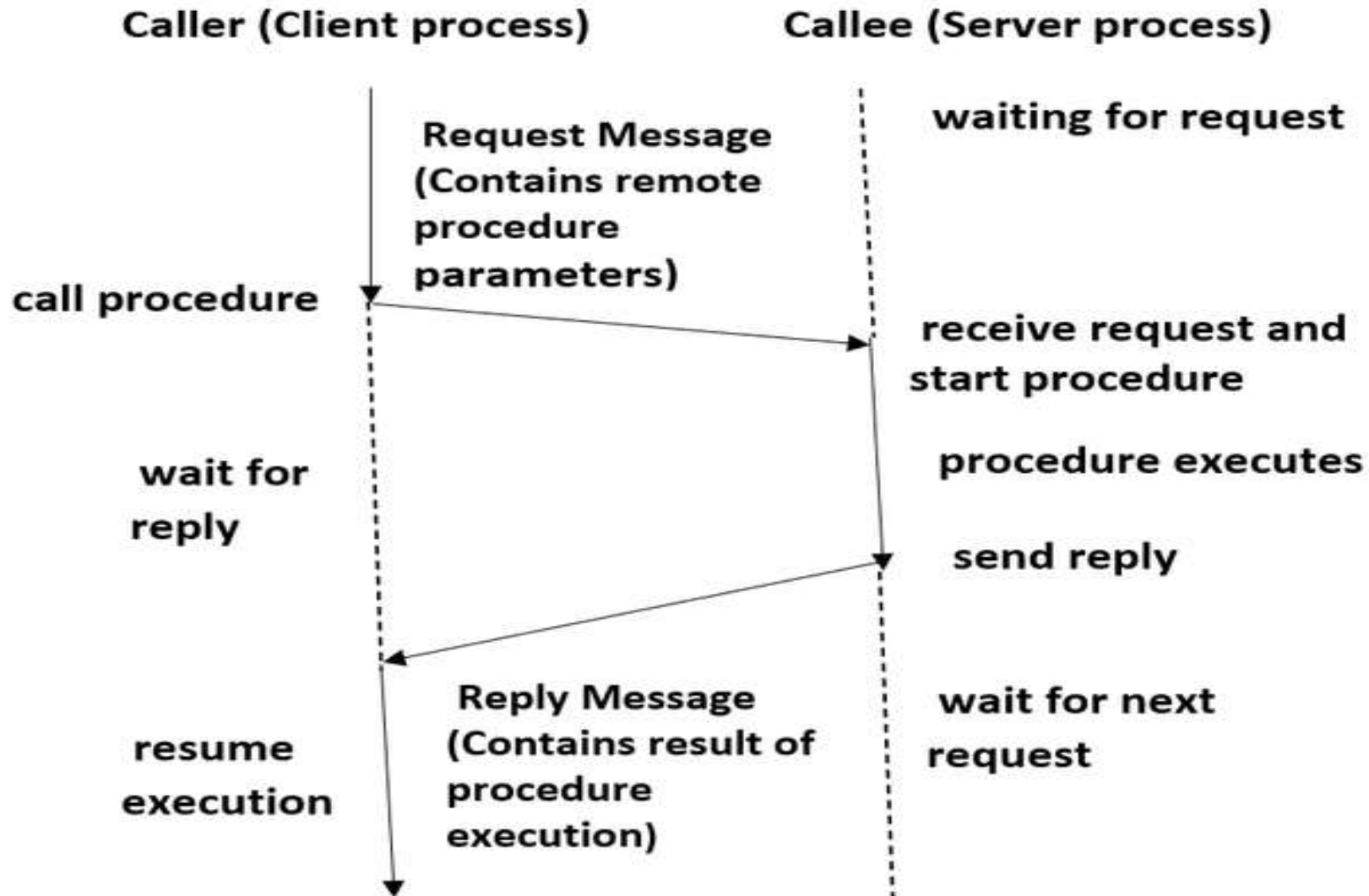
Remote Procedure Call (RPC)

- ✓ Remote Procedure Call (RPC) is a communication technology that is used by one program to make a request to another program for utilizing its service on a network without even knowing the network's details.
- ✓ A function call or a subroutine call are other terms for a procedure call.
- ✓ It is based on the client-server concept.
- ✓ The client is the program that makes the request, and the server is the program that gives the service.
- ✓ An RPC, like a local procedure call, is based on the synchronous operation that requires the requesting application to be stopped until the remote process returns its results.
- ✓ Multiple RPCs can be executed concurrently by utilizing lightweight processes or threads that share the same address space.
- ✓ Remote Procedure Call program as often as possible utilizes the Interface Definition Language (IDL), a determination language for describing a computer program component's Application Programming Interface (API).
- ✓ In this circumstance, IDL acts as an interface between machines at either end of the connection, which may be running different operating systems and programming languages.

Working Procedure for RPC Model:

- ✓ The process arguments are placed in a precise location by the caller when the procedure needs to be called.
- ✓ Control at that point passed to the body of the method, which is having a series of instructions.
- ✓ The procedure body is run in a recently created execution environment that has duplicates of the calling instruction's arguments.
- ✓ At the end, after the completion of the operation, the calling point gets back the control, which returns a result.
 - The call to a procedure is possible only for those procedures that are not within the caller's address space because both processes (caller and callee) have distinct address space and the access is restricted to the caller's environment's data and variables from the remote procedure.
 - The caller and callee processes in the RPC communicate to exchange information via the message-passing scheme.
 - The first task from the server-side is to extract the procedure's parameters when a request message arrives, then the result, send a reply message, and finally wait for the next call message.
 - Only one process is enabled at a certain point in time.
 - The caller is not always required to be blocked.
 - The asynchronous mechanism could be employed in the RPC that permits the client to work even if the server has not responded yet.
 - In order to handle incoming requests, the server might create a thread that frees the server for handling consequent requests.

Working Procedure for RPC Model:



Synchronization in Distributed Systems

- ✓ Distributed System is a collection of computers connected via a high-speed communication network.
- ✓ In the distributed system, the hardware and software components communicate and coordinate their actions by message passing.
- ✓ Each node in distributed systems can share its resources with other nodes. So, there is a need for proper allocation of resources to preserve the state of resources and help coordinate between the several processes. To resolve such conflicts, synchronization is used.
- ✓ Synchronization in distributed systems is achieved via clocks.
- ✓ The physical clocks are used to adjust the time of nodes.
- ✓ Each node in the system can share its local time with other nodes in the system.
- ✓ The time is set based on UTC (Universal Time Coordination).
- ✓ UTC is used as a reference time clock for the nodes in the system.
- ✓ .

Synchronization in Distributed Systems

✓ Clock synchronization can be achieved by 2 ways

❖ External and

❖ Internal Clock Synchronization

- **External clock synchronization** is the one in which an external reference clock is present. It is used as a reference and the nodes in the system can set and adjust their time accordingly.
- **Internal clock synchronization** is the one in which each node shares its time with other nodes and all the nodes set and adjust their times accordingly.

There are 2 types of clock synchronization algorithms: Centralized and Distributed.

- **Centralized** is the one in which a time server is used as a reference. The single time-server propagates its time to the nodes, and all the nodes adjust the time accordingly. It is dependent on a single time-server, so if that node fails, the whole system will lose synchronization. Examples of centralized are-Berkeley the Algorithm, Passive Time Server, Active Time Server etc.
- **Distributed** is the one in which there is no centralized time-server present. Instead, the nodes adjust their time by using their local time and then, taking the average of the differences in time with other nodes. Distributed algorithms overcome the issue of centralized algorithms like scalability and single point failure. Examples of Distributed algorithms are – Global Averaging Algorithm, Localized Averaging Algorithm, NTP (Network time protocol), etc.

Centralized clock synchronization algorithms suffer from two major drawbacks:

- They are subject to a single-point failure. If the time-server node fails, the clock synchronization operation cannot be performed. This makes the system unreliable. Ideally, a distributed system should be more reliable than its individual nodes. If one goes down, the rest should continue to function correctly.
- From a scalability point of view, it is generally not acceptable to get all the time requests serviced by a single-time server. In a large system, such a solution puts a heavy burden on that one process.
- Distributed algorithms overcome these drawbacks as there is no centralized time-server present. Instead, a simple method for clock synchronization may be to equip each node of the system with a real-time receiver so that each node's clock can be independently synchronized in real-time. Multiple real-time clocks (one for each node) are normally used for this purpose.